

GridPowerAgent: A Seeded, Reproducible Scenario Corpus for Power-System LLM Evaluation

Abstract—Evaluating large language models (LLMs) on power-system tasks requires observations that reveal what the model actually reads: several recent evaluations feed a plain-language description of the injected disturbance alongside the measurements and report near-perfect diagnosis. We present GridPowerAgent, a seeded, fully regenerable scenario corpus for power-system LLM evaluation: 15,000 scenarios across the IEEE 14, 39, and 118 bus systems, ten injected disturbance classes, 9.54 million correlated measurements, per-scenario iterative weighted-least-squares (WLS) state estimates, dual-axis reference keying, a reproducible 41-scenario failure registry, and an executable pandapower tool back-end. The corpus ships three observation channels that separate what a model reads from what it reasons about: a plain-language event description, EMS-grade telemetry, and raw summary state. An illustrative study—one seeded 140-scenario draw, two models, single temperature-zero runs with exact denominators—demonstrates the corpus’s central measurement: description-only diagnosis is 100%, the description is the answer key; from telemetry alone the API model reaches 93.9% while a 4-bit local model drops to 43.8% (20-scenario probe); from raw summary state the API model collapses to 8.0% while a gradient-boosted classifier on identical features reaches 64.3%. The corpus, the estimation pipeline, the keying, and the registry are the contribution; the study illustrates what they enable, not a general finding about grid-LLM evaluation.

Keywords—*benchmark corpus, cause-keyed labels, iterative WLS, LLM evaluation, observation channels, pandapower, power systems, reproducibility, scenario generation, state estimation.*

I. INTRODUCTION AND RELATED WORK

LLMs are increasingly proposed for power-system operations: reading grid states, classifying disturbances, selecting analysis tools, and advising operators [1], [2], [3], [4]. Evaluating such claims requires a test corpus with three properties that published evaluations—LLM dispatch benchmarks [5], [6] and reinforcement-learning grid testbeds [7], [8] alike—rarely provide: the labels are derived by a transparent, reproducible rule system; every scenario carries physical ground truth; and the observation given to the model is separated from the information used to grade it. This paper presents such a corpus and the evaluation harness built around it.

A seeded corpus changes what an evaluation can claim. Fixed draws mean every model, prompt version, and observation channel is scored on byte-identical inputs, so paired comparisons are exact rather than resampled. Regeneration from seeds makes label provenance auditable: every label traces to an injection record and a solver run, and leakage audits (did the observation disclose the label?) are plain joins over the 54 scenario fields. Physical ground truth also enables non-LLM baselines—the classifiers in Sec. III-A that

bound what information each channel carries—which pure text benchmarks cannot run. Finally, a released corpus turns evaluation into regression testing: new models are scored against prior runs on identical scenarios instead of re-annotated prompts.

This paper contributes the artifact, not a model ranking. The contributions are: (1) a seeded, fully regenerable corpus of 15,000 scenarios across the IEEE 14, 39, and 118 bus systems (3,000/5,000/7,000; 300–700 per disturbance class), generated from seeded injections over 16,000 operating points, each carrying pre- and post-event power-flow ground truth [9], 9.54 million correlated measurements, and physics flags; (2) a per-scenario measurement and iterative-WLS state-estimation pipeline with a disclosed noise model, shipped estimates, and audited accuracy; (3) a dual-axis reference keying that keeps outcome-keyed and cause-keyed labels scoreable side by side, plus a reproducible registry of the 41 IEEE-39 injections that produce unresolvable islands; (4) three separated observation channels with an executable pandapower tool back-end and a closed-loop protocol; and (5) an explicitly illustrative two-model, one-draw study of the observation channels that demonstrates what the corpus measures. The corpus, the estimation pipeline, the keying, and the registry stand on their own; nothing in this paper argues a general claim about grid-LLM performance.

A. LLM Grid Evaluations

LLM grid evaluations report strong task performance: ElecBench evaluates dispatch prompting [5] and GAIA fine-tunes a dispatch model [6]; both grade answers to prompts that describe the event in plain language.

B. Agentic Grid Systems

Agentic grid systems couple LLMs to grid environments—Grid-Agent [3] for control, X-GridAgent [4] for analysis—and report end-to-end system behavior without shipping an evaluation corpus.

C. Reinforcement-Learning Grid Testbeds

Reinforcement-learning testbeds (L2RPN [7], Grid2Op [8]) ship reproducible scenario sets with simulator ground truth, but their interfaces target RL topology control rather than LLM-facing observation channels.

D. Positioning Against Existing Artifacts

Table I positions GridPowerAgent against these artifacts on seven criteria: seeded regenerable scenarios (Seed), per-scenario power-flow ground truth (Phys.), LLM-facing obser-

TABLE I. RELEASED ARTIFACTS COMPARED

Artifact	Seed	Phys.	Obs.	Label	WLS	Tools	LLM
ElecBench [5]	—	—	—	—	—	—	✓
GAIA [6]	—	—	—	—	—	—	✓
Grid-Agent [3]	—	—	—	—	—	—	✓
X-GridAgent [4]	—	—	—	—	—	—	✓
L2RPN [7]	✓	✓	—	—	—	—	—
Grid2Op [8]	✓	✓	—	—	—	—	—
GridPowerAgent (ours)	✓	✓	✓	✓	✓	✓	✓

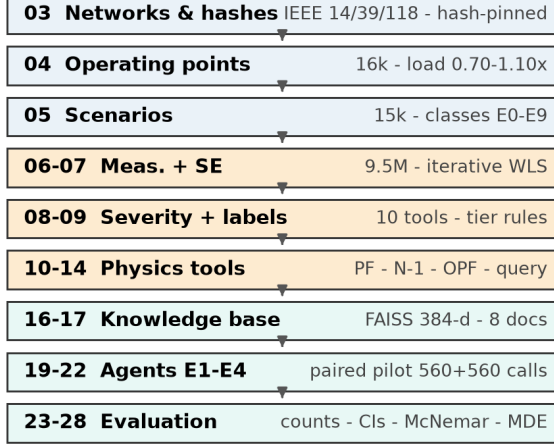


Fig. 1. Corpus generation pipeline (stages 03–28).

vation channels (Obs.), cause-keyed reference labels (Label), shipped iterative-WLS state estimates (WLS), an executable pandapower tool back-end (Tools), and reported LLM performance (LLM; — = not part of the released artifact). GridPowerAgent combines all seven in one artifact.

II. METHODOLOGY

A. Networks, Operating Points, and Injected Classes

Fig. 1 overviews the generation pipeline. Networks use pandapower [10] IEEE 14/39/118 cases [11] with tuned thermal limits to make compound events observable (14: 3% line / 4% transformer; 118: 6%; 39: nameplate). Limits are disclosed per system and deliberately not comparable across systems. Operating points sweep load 0.70–1.10 $\times$ with $\pm 5\%$ bus-local noise, renewable fractions, and storage state of charge 0.15–0.85 (20 MW/40 MWh at bus 9 on IEEE-14). Topology hashes pin all three networks. Ten classes are injected: E0 Normal; E1 load surge, E2 load drop, E3 line outage, E4 generator outage, E5 renewable ramp (cause classes); E6 undervoltage, E7 overvoltage, E8 thermal overload (outcome classes, physically iterated to target severity); E9 compound.

B. Composition

Table II summarizes the corpus. Scenarios inject the ten classes with 300/500/700 per class on 14/39/118; replay over 40 draws confirms exact determinism. Fig. 2 shows the class composition of the three systems.

Every scenario row is self-describing (54 fields). For example, IEEE14_SCN_000901 (E3, transmission-line outage) draws operating point IEEE14_OP_002268 (load scale 0.77, 194.9 MW system load), injects a line_outage on target line_6_12, and records matched pre/post quantities: minimum

TABLE II. CORPUS SUMMARY PER SYSTEM

Case	OPs	Scenarios	Per class	Measurements
14 (ref)	4k	3,000	300	361k
39	5k	5,000	500	1.50M
118	7k	7,000	700	7.68M

voltage 0.9865 $\rightarrow$ 0.9851 pu at bus 14, with no resulting limit violations on this draw. Injection descriptors (mechanism, scope, targets, magnitude), the operating point, post-event power-flow quantities, violation counts and flags, and per-quantity deltas all ship in one row, so labeling, subsetting, and leakage audits are plain joins.

Under the cause-keyed default, class counts shift from the 300-per-class injection design because the E6/E8 scenarios re-key to the mechanism that produced them: the IEEE-14 labels are 300 each for E0–E2, E5, and E7; 579 for E3; 329 for E4; and 592 for E9.

C. Severity Ladders and Compound Design

The cause classes (E1–E5, E9) inject a mechanism and record whatever state results. The outcome classes are built the opposite way: a physical disturbance is applied, the power flow is solved, and the outcome is detected—if the disturbance did not produce the target state, the scenario is escalated along a severity ladder or discarded, so for every E6/E8 scenario the solver, not the author, decided the voltage or loading. This is also why E7 is generated through reactive mechanisms (excitation set-point shifts rather than load/renewable manipulation): on the stiff IEEE-14 system, injections cannot lift the bus-voltage maximum above 1.05 pu at all (the measured ceiling is 1.0347 pu at minimum load with both renewables at 100% and the battery discharging). Compound scenarios apply exactly two component mechanisms in order and record the union of their targets. The measured severity envelope on IEEE-14 bounds the corpus: every single-line outage converges and none islands the system, the worst branch loading reaches 228% (line_1_2 outage), and the deepest voltage sag reaches 0.9425 pu (line_9_14 outage).

D. Measurements and Iterative WLS State Estimation

Each scenario carries a fixed meter complement generated from the noise-free power-flow solution: 118 meters per IEEE-14 scenario (14 bus-voltage magnitudes with $\sigma_V = 0.003$ pu, 76 branch-end P/Q, 28 bus injections), 301 per IEEE-39 scenario, and 1,098 per IEEE-118 scenario—9.54 million readings corpus-wide. Branch-end and injection meters both use $\sigma_P = \max(0.0075|S_{\text{true}}|, 0.05)$ MVA, and every reading ships its true value, noisy measurement, σ , and standardized z -score. The state estimate supplied to the agent is produced by an iterative AC-WLS estimator [12], [13] (pandapower.estimate) over these measurements, with branch meters of outaged elements excluded: the estimator converges on 140/140 pilot scenarios with voltage RMSE 0.0064 pu against post-event truth, and the under-voltage flag agrees with truth in 75% of scenarios—marginal violations can vanish under estimation, which the closed-loop evaluation in Sec. III-C probes directly.

TABLE III. ISLANDING FAILURE REGISTRY (IEEE-39)

Switched element	Islanded buses	Scenarios
line_16_19	19, 20, 33, 34	24
line_23_36	36	17
Total		41

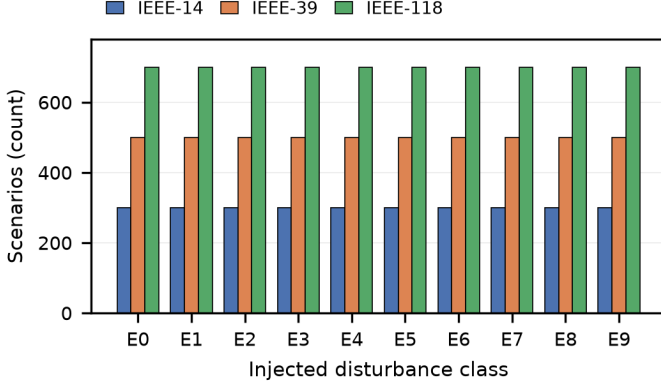


Fig. 2. Class composition of the three systems (scenarios per injected class).

E. Failure Registry

Two IEEE-39 switching injections disconnect load pockets and produce unresolvable islands (41 scenarios, NaN post-event voltages). Their identifiers and causes ship with the corpus (case39_nan_scenarios.csv) and the scenarios are excluded from evaluation; the resolution plan is an islanding-aware contingency pre-filter in the next corpus revision. Table III lists the registry in full.

F. Dual-Axis Reference Keying

Every scenario ships two label versions: the original v1 keying labeled the two outcome classes (E6 undervoltage, E8 thermal overload) by post-event outcome, and the revised v2 keying labels every scenario by its injected mechanism. The outcome information remains in the per-scenario physics flags, so both axes stay scoreable. The v1 keying proved consequential—models graded under it scored zero on E6/E8 while describing the injecting cause correctly—so the cause-keyed version is the evaluation default, and the failure mode is reproducible by design rather than latent.

G. Artifact Inventory and Reproducibility

Per system, the released artifact contains the scenario table (CSV and JSONL), pre/post power-flow ground truth, the full measurement set with true value, noisy measurement, σ , and z -score per meter, the WLS state estimates for the pilot draws, both reference-label axes, and the islanding registry; the tool back-end, the FAISS knowledge base, and all run CSVs ship alongside. Networks are pinned by topology hash, operating points and injections are seeded draws, and 40-draw replay reproduces the scenarios byte-identically. Regeneration requires only pandapower and the seeded generation scripts; no number in this paper depends on unreleased data.

H. Tools and Knowledge Base

A pandapower back-end exposes five executable operations: full AC power flow, N-1 contingency testing on a named

branch, OPF [14], and grid queries (topology, limits, equipment). A retrieval-augmented knowledge base [15]: FAISS [16] chunks eight operational documents (thermal/voltage limits, contingency procedure, storage/equipment, topology) into 384-d sentence-transformer embeddings [17], retrieving $k = 3$ chunks with citations.

I. Using the Corpus

The generation choices imply an evaluation protocol. Score against the cause-keyed axis and report which axis was used: the outcome-keyed variant ships so that the keying failure of Sec. II-F is reproducible by design rather than latent. Report the observation channel with every number: the three channels are separate prompt-side payloads over byte-identical scenarios, so channel and model effects never have to be disentangled after the fact. Prefer paired seeded draws: regression-testing a new model against prior runs on identical inputs makes comparisons exact rather than resampled. Exclude the 41 registry scenarios (or handle islands explicitly)—their identifiers ship with the corpus. Run non-LLM classifier baselines on each channel before interpreting model scores: a channel that no classifier can separate cannot be expected to inform any model. In closed-loop use, grade element construction separately from class match: the two coincide on single-mechanism events and part on compounds (Sec. III-C).

J. Prompt Contract and Tool Configurations

The prompt contract is fixed across all studies: the system prompt fixes the persona and tool rules, the user message carries the observation-channel payload, and the answer must be a JSON object naming the diagnosis class—and, in the closed-loop condition, the switched element. Four tool configurations ship with the harness—E1 bare prompt, E2 with retrieved knowledge-base chunks, E3 with executable tools, E4 with both—so retrieval and tool effects can be ablated under an identical contract; retrieval-augmented configurations append the $k = 3$ retrieved chunks with citations to the user message.

K. Observation Channels

Three prompt-side conditions probe what diagnosis measures: description-only (the plain-language event statement, no grid numbers), telemetry (an observed-evidence block with no event statement), and full (both). The telemetry block is fixed and fully disclosed—five lines: which branches are open and which generators are offline (by element name); which demand elements changed; which renewable elements changed; system-wide pre→post deltas for load, slack generation, and losses; and deltas for $V_{\min}$, $V_{\max}$, and peak loading. It contains observable facts only: no injected intent, no natural-language description. A WLS-estimated variant of the telemetry channel replaces the true post-event values in this block with estimated ones. Table IV shows the complete block for the case-study scenario of Sec. III-C: the demand-side component is fully legible here (eleven changed demand elements, $\Delta\text{load} +40.28$ MW)—exactly the information the closed-loop status table omits.

TABLE IV. CASE-STUDY TELEMETRY BLOCK

Block	Content
Switching	branches OPEN: line_3_4; no generators offline
Changed demand	load_bus_2, load_bus_3, load_bus_4, load_bus_5, load_bus_6, load_bus_9, load_bus_10, load_bus_11, load_bus_12, load_bus_13, load_bus_14
Changed renewables	none
System deltas	Δ load: +40.28 MW; Δ slack generation: +45.77 MW; Δ losses: +5.49 MW
Voltage deltas	$\Delta V_{\min}$: -0.0118 pu; $\Delta V_{\max}$: +0.0000 pu; Δ peak loading: +0.357 pp

L. Models

The illustrative study of Sec. III runs two models under this contract: a lightweight API model (Gemini 3.5 Flash Lite, temperature 0, ≤ 512 output tokens) and a small 4-bit-quantized open-weight model [18], [19] (Gemma 4 E4B-it [20], served locally from a GGUF build [21]; temperature 0, $\leq 1,024$ tokens, reasoning channel parsed with content priority). The API tier is deliberately lightweight: the study characterizes a local deployment against a budget API tier, not against frontier models.

M. Closed-Loop Tool Protocol

The closed-loop condition embeds the agent in a tool loop in the reasoning-plus-acting [22] and tool-calling [23] style: it proposes structured calls with arguments, each call is validated against the network (referenced elements must exist) and executed by the real pandapower back-end, returned measurements re-enter the conversation, at least two executed calls are required before an answer is accepted, and the answer must name the switched element. The closed-loop observation differs from the one-shot telemetry block: the agent sees a per-element status table (each branch as name | in-service | loading %, each generator as name | offline state) and, from tools, system summaries (voltage extremes, peak loading, overload list)—switching state and loading, but no demand-side deltas.

N. Metrics

Diagnosis correctness is exact class match against the cause-keyed reference; tool-call validity requires referenced elements to exist and calls to execute; closed-loop episodes additionally receive a construction grade (did the answer name the true switched element). For paired designs the harness reports the minimum detectable difference (MDE) at $\alpha = 0.05$, power 0.80, $(z_{0.975} + z_{0.80})\sqrt{p_d/n}$, where p_d is the paired discordance rate. All study numbers are exploratory single temperature-zero runs with exact denominators.

III. RESULTS AND DISCUSSION

This section demonstrates what the corpus measures; it does not establish a model ranking. Scope: one seeded draw of 140 scenarios per condition (20 for the local telemetry probe), the two models of Sec. II-J, single temperature-zero runs with exact denominators. We read the numbers as evidence about the corpus’s instruments—that its channels separate description-reading from state-based diagnosis—not as findings about model classes.

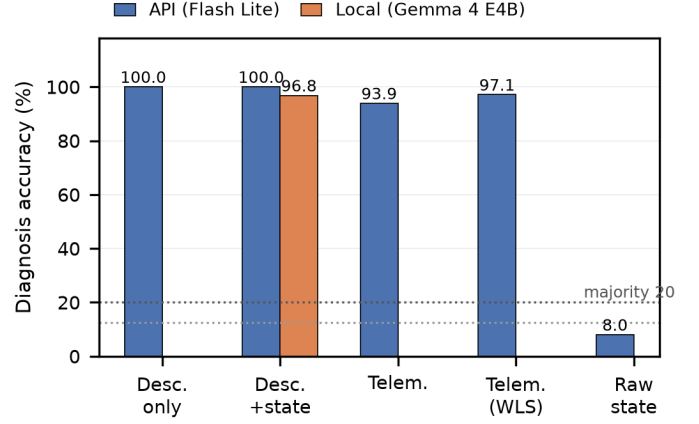


Fig. 3. Illustrative study: diagnosis by observation channel and tier (one seed draw, two models). Dotted lines: majority (20.0%) and random (12.5%) baselines.

TABLE V. DIAGNOSIS ACCURACY (%) BY OBSERVATION CONDITION

Condition	API	Local	GBM
Description only	100.0	—	—
Description + state	100.0	96.8	—
Telemetry	93.9	—	100.0
Telemetry (WLS est.)	97.1	—	—
Raw state only	8.0	—	64.3

A. Observation Channels and Baselines

With only the plain-language event description available, the API model diagnoses 100.0% (560 scenarios): reading the description is sufficient, confirming that the description functions as an answer key when present. Table V and Fig. 3 report diagnosis by channel. From telemetry alone the API model reaches 93.9%; replacing the true post-event state with the WLS estimate leaves it at 97.1%. From raw summary state it collapses to 8.0%, answering E7 for most scenarios. The signal exists: a gradient-boosted classifier on identical features reaches 64.3% from raw state and 100.0% from telemetry (classifiers fitted on non-pilot scenarios, evaluated on the pilot draw), so the raw-state shortfall is an encoding limit, not an information limit—exactly the distinction the corpus is built to expose. The local model shows the same logic from the other side: 96.8% with description+state, 43.8% from telemetry alone (20-scenario probe); the two readings separate only under the realistic channel. Because the channels are prompt-side payloads scored on byte-identical draws, every number above is exactly paired: the corpus makes the observation channel a first-class reported variable by construction rather than by discipline.

B. Cross-Network Draw

A second seeded draw on IEEE-39 (140 scenarios) runs under the identical prompt contract, taxonomy rules, and scoring code: on the description+state channel the API model diagnoses 140/140 and the local model 134–139/140 across configurations. Nothing was re-tuned for the larger system.

C. Closed-Loop Channel Illustration

In the closed-loop condition the agent must name the switched element and chain at least two real tool executions.

On 107 scenarios the API model reaches 30.8% construction-graded diagnosis, with 71.9% element construction on the 64 switched-element scenarios; all 219 proposed calls validated and executed (219/219), averaging 2.05 calls and 3.59 turns per episode.

Episode IEEE14_SCN_002945 shows what the construction grade adds. The scenario is a compound E9 event: an outage of line_3_4 plus a 15% demand increase spread over eleven buses. In three turns the agent issued two executed grid_query calls, located the out-of-service branch, and named line_3_4—the true target, so construction succeeds. Its final answer was E3: the status table shows the outage clearly, and its stated reason reads “branch line_3_4 is OUT_OF_SERVICE with demand unchanged,” so the distributed demand-side component went unnoticed. The pattern is systematic: all 21 switched episodes that name the correct element but answer the wrong class are E9 compound scenarios, and class agreement on switched episodes is 25/64 (39.1%) against 33/107 (30.8%) overall.

The corpus-level reading is an encoding measurement, not a model verdict: the closed-loop status table and tool summaries carry switching state and loading but no demand-side deltas, so the demand component of a compound event is unobservable in that channel—while the one-shot telemetry block lists changed demand elements explicitly (Table IV). The corpus ships both encodings precisely so that this difference is measurable. Construction and class grades coincide when the named element’s mechanism is the whole event; compound events are where they part, and a class match alone would hide that the agent was right about what switched.

D. Limitations and Threats to Validity

Study scope: every cell is a single temperature-zero run on one seed draw: reported differences carry no sampling variance, and the paired MDE bounds only what this design could detect. The pilots cover 140 scenarios per condition (20 for the local telemetry probe), so rates on rare classes are coarse. The two models are a budget API tier and a 4-bit-quantized local model; nothing here characterizes frontier models. The automated judge flagged 0 API and 3 local responses across 1,120 paired rows; given known LLM failure modes [24] and miscalibration [25], these are judge agreements, not expert-annotated ground truth. Latency (1.1 s per API call versus 44–49 s locally) reflects single-stream local serving, not a tuned deployment.

Corpus scope: the corpus covers three static test systems with snapshot disturbances; dynamic (time-series) events are future work. Thermal limits are tuned per system to make compound events observable and are deliberately not comparable across systems. The 41 islanding scenarios are excluded from evaluation pending an islanding-aware contingency pre-filter. The closed-loop construction metric is defined relative to the shipped observation format; a richer encoding would shift it. The outcome-keyed axis ships for reproducibility of the keying result but is not exercised by the models reported here.

IV. CONCLUSION

GridPowerAgent contributes what power-system LLM evaluation currently lacks: a seeded, regenerable 15,000-scenario corpus with physical ground truth, a per-scenario iterative-WLS estimation pipeline, dual-axis reference keying, a reproducible islanding failure registry, three observation channels that separate description-reading from state-based diagnosis, and an executable tool back-end with a closed-loop protocol. Because every scenario regenerates from seeds and every row carries its own ground truth, the corpus turns grid-LLM evaluation into exact paired regression testing rather than re-annotated prompting.

The illustrative study shows the kind of measurement the corpus makes routine: the observation channel, not the model, decides what is diagnosable—description-only diagnosis is trivially perfect, telemetry-only separates the two models tested, and raw-state diagnosis collapses while the classifier baselines show the missing performance is an encoding limit, not an information limit. In closed-loop use, an agent can name the switched element correctly and still miss the event, and a class-match metric alone would report both failures identically. Future work includes expert κ annotation of a labeled subset, a powered 600-scenario sweep, dynamic (time-series) disturbances, the islanding-aware contingency pre-filter for the 41 registry cases, richer closed-loop observation encodings, and full-precision local serving.

ACKNOWLEDGMENT

The authors acknowledge writing-support tools (OpenAI ChatGPT, Grammarly, and QuillBot) for grammar correction, readability improvement, language refinement, and sentence restructuring. All technical content, analysis, interpretations, and research contributions are those of the authors.

REFERENCES

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman *et al.*, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [2] S. Majumder, L. Dong, F. Doudi, Y. Cai, C. Tian, D. Kalathil, K. Ding, A. A. Thatte, N. Li, and L. Xie, “Exploring the capabilities and limitations of large language models in the electric energy sector,” *Joule*, vol. 8, pp. 1544–1549, 2024.
- [3] Y. Zhang, A. M. Saber, A. Youssef, and D. Kundur, “Grid-agent: An LLM-powered multi-agent system for power grid control,” *arXiv preprint arXiv:2508.05702*, 2025.
- [4] Y. Wen and X. Chen, “X-GridAgent: An LLM-powered agentic AI system for assisting power grid analysis,” *arXiv preprint arXiv:2512.20789*, 2025.
- [5] X. Zhou, H. Zhao, Y. Cheng, Y. Cao, G. Liang, G. Liu, W. Liu, Y. Xu, and J. Zhao, “ElecBench: A power dispatch evaluation benchmark for large language models,” *arXiv preprint arXiv:2407.05365*, 2024.
- [6] Y. Cheng, H. Zhao, X. Zhou, J. Zhao, Y. Cao, and C. Yang, “A large language model for advanced power dispatch,” *Scientific Reports*, vol. 15, p. 8925, 2025.
- [7] A. Marot, B. Donnot, G. Dulac-Arnold, A. Kelly, A. O’Sullivan, J. Viebahn, M. Awad, I. Guyon, and B. Donon, “Learning to run a power network challenge for training topology controllers,” *Electric Power Systems Research*, vol. 189, p. 106635, 2020.
- [8] B. Donnot *et al.*, “Grid2Op: A testbed platform to model sequential decision making in power systems,” <https://github.com/rte-france/grid2op>, 2020.
- [9] A. J. Wood, B. F. Wollenberg, and G. B. Sheblé, *Power Generation, Operation, and Control*, 3rd ed. Wiley, 2014.
- [10] L. Thurner, A. Scheidler, F. Schäfer, J.-H. Menke, J. Dolichon, F. Meier, S. Meinecke, and M. Braun, “pandapower—an open-source Python tool for convenient modeling, analysis, and optimization of electric power systems,” *IEEE Transactions on Power Systems*, vol. 33, no. 6, pp. 6510–6521, 2018.
- [11] R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas, “MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education,” *IEEE Transactions on Power Systems*, vol. 26, no. 1, pp. 12–19, 2011.
- [12] A. Abur and A. Gómez-Expósito, *Power System State Estimation: Theory and Implementation*. Marcel Dekker, 2004.
- [13] A. Monticelli, *State Estimation in Electric Power Systems: A Generalized Approach*. Springer, 1999.

- [14] S. Frank, I. Steponavice, and S. Rebennack, "Optimal power flow: a bibliographic survey I," *Energy Systems*, vol. 3, no. 3, pp. 221–289, 2012.
- [15] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [16] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.
- [18] T. Dettmers and L. Zettlemoyer, "The case for 4-bit precision: k-bit inference scaling laws," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 10 775–10 804.
- [19] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *International Conference on Learning Representations (ICLR)*, 2023.
- [20] Gemma Team, Google DeepMind, "Gemma 4 technical report," *arXiv preprint arXiv:2607.02770*, 2026.
- [21] G. Gerganov and contributors, "llama.cpp: LLM inference in C/C++," github.com/ggml-org/llama.cpp, 2023.
- [22] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.
- [23] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [24] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [25] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.